

DSA Pattern Series - Lecture 5

Two Pointer Pattern: Dutch National Flag & Triplet Sum Smaller

Overview

This lecture covers the final two problems in the Two Pointer Pattern series:

1. **Triplet Sum Smaller** (LeetCode 259)
2. **Dutch National Flag Algorithm** (Sort Colors - LeetCode 75)

Both problems are critical for interview preparation and demonstrate advanced application of the two-pointer technique.

Part 1: Triplet Sum Smaller

Problem Statement

Question: Given an array of integers and a target sum, find the **count of triplets** where the sum is **less than the target sum**.

Example: - **Input:** arr = [-1, 0, 1, 3], target = 2 - **Output:** 3 - **Explanation:** Triplets with sum less than 2: - (-1, 0, 1) → sum = 0 < 2 - Other valid triplets exist with indices that satisfy the condition

Key Constraints

- Array elements are **distinct** (no duplicates)
- Find **count** of triplets, not the triplets themselves
- Time complexity should be **O(n²)** using Two Pointer approach

Solution Approach

Step 1: Sort the Array

Original: [-1, 0, 1, 3]

Sorted: [-1, 0, 1, 3]

Step 2: Fix One Element and Use Two Pointers

- Fix index **i** from 0 to n-2
- For each fixed **i**:
 - Place **left** pointer at **i+1**
 - Place **right** pointer at **n-1**

Step 3: Two Pointer Logic Key Insight: If $\text{arr}[i] + \text{arr}[\text{left}] + \text{arr}[\text{right}] < \text{target}$, then ALL elements between `left` and `right` will also satisfy the condition because the array is sorted.

```
If sum < target:
    - arr[i] + arr[left] + arr[right] < target
    - arr[i] + arr[left] + arr[right-1] < target    (because arr[right-1] < arr[right])
    - arr[i] + arr[left] + arr[right-2] < target
    ...
```

Count all these combinations: `answer += (right - left)`
Then increment left pointer

If sum $\geq$ target: - Decrement right pointer (need smaller sum)

Pseudocode

```
function countTripletsSmallerSum(arr, target):
    sort(arr)
    count = 0

    for i from 0 to n-2:
        left = i + 1
        right = n - 1

        while left < right:
            sum = arr[i] + arr[left] + arr[right]

            if sum < target:
                count += (right - left) // All combinations count
                left++
            else:
                right--

    return count
```

Time & Space Complexity

- **Time Complexity:** $O(n^2)$
 - Sorting: $O(n \log n)$
 - Nested loops: $O(n^2)$
 - Overall: $O(n^2)$
- **Space Complexity:** $O(1)$
 - Only using pointers, no extra data structures

Why This Matters

This is a **twisted version** of the 3Sum problem. Instead of finding triplets with exact target sum, we count triplets with sum **less than** target. The mathematical insight about counting ranges is crucial for interview success.

Companies asking: Google, Amazon, Flipkart, Microsoft

Part 2: Dutch National Flag Algorithm

Problem Statement

Question: Given an array containing only 0, 1, and 2, sort the array **in-place** without using sorting algorithms.

Example: - **Input:** [2, 0, 2, 1, 0, 0, 1, 2] (indices 0-7) - **Output:** [0, 0, 0, 1, 1, 2, 2, 2]

Problem Context

The algorithm is named after the **Dutch flag** () which has three horizontal color stripes (red, white, blue), representing the three regions we need to create in the array.

Key Requirements

In-place: No extra space allowed ($O(1)$ space)

Without sorting: Cannot use `sort()` function

Single pass preferred: Optimal solution uses only one pass

Approaches

Approach 1: Brute Force - Counting Method (Two Pass) Step 1:

Count occurrences of each number

Array: [2, 0, 2, 1, 0, 0, 1, 2]

Count of 0s: 3

Count of 1s: 2

Count of 2s: 3

Step 2: Overwrite array based on counts

First 3 positions: write 0s $\rightarrow$ [0, 0, 0, _, _, _, _, _]

Next 2 positions: write 1s $\rightarrow$ [0, 0, 0, 1, 1, _, _, _]

Last 3 positions: write 2s $\rightarrow$ [0, 0, 0, 1, 1, 2, 2, 2]

Complexity: - Time: $O(2n) = O(n)$ (two passes) - Space: **$O(1)$** (only three variables)

Interview Note: This approach is acceptable and hard to argue against in interviews.

Approach 2: Optimal - Three Pointer (Single Pass) This is the **Dutch National Flag Algorithm** - the most elegant solution.

Three Pointer Strategy

Concept: Divide the array into three regions

```
[0 to low-1]    → All 0s (sorted )
[low to mid-1]  → All 1s (sorted )
[mid to high]   → Unknown (needs sorting)
[high+1 to n-1] → All 2s (sorted )
```

Initial state:

```
low = 0, mid = 0, high = n-1
```

Goal: Move mid from 0 to n-1, placing elements in correct regions

Stop when: mid > high

Algorithm Flow

```
for each element at arr[mid]:

    if arr[mid] == 0:
        // 0 belongs in [0, low)
        swap(arr[mid], arr[low])
        low++
        mid++ // arr[low] was 1, now 0 is placed

    if arr[mid] == 1:
        // 1 belongs in [low, mid)
        mid++ // Already in correct region

    if arr[mid] == 2:
        // 2 belongs in [high+1, n)
        swap(arr[mid], arr[high])
        high-- // Don't increment mid - check swapped element
```

Why Don't We Increment Mid When Swapping with High?

Key Insight: When we swap with high, we're moving a 2 to the right. But what comes from high is **unknown** (could be 0, 1, or 2). We need to reprocess it.

Example:

```
mid = 3, arr[mid] = 2, high = 8, arr[high] = 0
```

```
After swap: arr[mid] = 0
```

```
// Now we need to handle this 0 in the next iteration
```

```
// If we increment mid, we skip processing this 0!
```

Detailed Example Walkthrough

```
Input: [2, 0, 2, 1, 0, 0, 1, 2]
```

```
Index: 0 1 2 3 4 5 6 7
```

```
Initial: low=0, mid=0, high=7
```

```
Iteration 1: arr[mid=0] = 2
```

```
Swap arr[0] with arr[7]: [2, 0, 2, 1, 0, 0, 1, 2]
```

```
high-- → high=6
```

```
Iteration 2: arr[mid=0] = 2 (swapped from arr[7])
```

```
Swap arr[0] with arr[6]: [1, 0, 2, 1, 0, 0, 2, 2]
```

```
high-- → high=5
```

```
Iteration 3: arr[mid=0] = 1
```

```
mid++ → mid=1
```

```
Iteration 4: arr[mid=1] = 0
```

```
Swap arr[1] with arr[0]: [0, 1, 2, 1, 0, 0, 2, 2]
```

```
low++ → low=1
```

```
mid++ → mid=2
```

```
... (continue until mid > high)
```

```
Final: [0, 0, 0, 1, 1, 2, 2, 2]
```

```
       [0-2] [3-4] [5-7]
```

```
       ↓   ↓   ↓
```

```
       0s   1s   2s
```

Complexity Analysis

- **Time Complexity:** $O(n)$ (single pass)
- **Space Complexity:** $O(1)$ (only three pointers)

Why it's optimal: One single pass through the array - can't do better than $O(n)$.

Pseudocode

```
function dutchFlagSort(arr):
    low = 0
    mid = 0
    high = len(arr) - 1

    while mid <= high:
        if arr[mid] == 0:
            swap(arr[mid], arr[low])
            low++
            mid++

        else if arr[mid] == 1:
            mid++

        else: // arr[mid] == 2
            swap(arr[mid], arr[high])
            high--
```

Code Implementation

Python

```
def sortColors(arr):
    low, mid, high = 0, 0, len(arr) - 1

    while mid <= high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 1:
            mid += 1
        else: # arr[mid] == 2
            arr[mid], arr[high] = arr[high], arr[mid]
            high -= 1

    return arr
```

JavaScript

```
function sortColors(arr) {
    let low = 0, mid = 0, high = arr.length - 1;

    while (mid <= high) {
        if (arr[mid] === 0) {
```

```

        [arr[low], arr[mid]] = [arr[mid], arr[low]];
        low++;
        mid++;
    } else if (arr[mid] === 1) {
        mid++;
    } else {
        [arr[mid], arr[high]] = [arr[high], arr[mid]];
        high--;
    }
}

return arr;
}

```

Java

```

public void sortColors(int[] arr) {
    int low = 0, mid = 0, high = arr.length - 1;

    while (mid <= high) {
        if (arr[mid] == 0) {
            swap(arr, low++, mid++);
        } else if (arr[mid] == 1) {
            mid++;
        } else {
            swap(arr, mid, high--);
        }
    }
}

private void swap(int[] arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}

```

C++

```

void sortColors(vector<int>& arr) {
    int low = 0, mid = 0, high = arr.size() - 1;

    while (mid <= high) {
        if (arr[mid] == 0) {
            swap(arr[low++], arr[mid++]);
        } else if (arr[mid] == 1) {
            mid++;
        }
    }
}

```

```

        } else {
            swap(arr[mid], arr[high--]);
        }
    }
}

```

Comparison: Three Pointer Approaches

Approach	Passes	Time	Space	Interview Friendly
Brute Force (Counting)	2	$O(n)$	$O(1)$	Yes
Three Pointer (DNF)	1	$O(n)$	$O(1)$	Better

Why Three Pointer is “better”: Shows pattern recognition and algorithmic insight. Demonstrates understanding of pointer manipulation.

Pattern Recognition for Two Pointer

When to Use Two Pointer Pattern?

Array is sorted (or can be sorted)
Need to find pairs/triplets with conditions
In-place modifications required
Opposite ends need adjustment based on condition
Problems ask for: Count, arrangement, or partition

Two Pointer Variations

Variation	Use Case	Example
Low-High	Opposite ends movement	Container with Most Water, Palindrome
Same Direction	Slow-Fast pointers	Remove Duplicates, Partition
Fixed Element	One element fixed	3Sum, 3Sum Smaller
Three Pointers	Three regions needed	Dutch National Flag

Interview Preparation Notes

Expected Questions

Interviewer might ask:

1. **Why sort first?**
 - Answer: Enables two-pointer logic; enables counting ranges
2. **Can we do it without sorting?**
 - Answer: For Triplet Smaller: No, we need sorted order for the range counting logic
 - Answer: For Dutch Flag: Yes, using three pointers (no sorting)
3. **Why don't we increment mid after swapping with high?**
 - Answer: Because we don't know what value came from high; need to reprocess it
4. **What if array has other values besides 0, 1, 2?**
 - Answer: This algorithm only works for exactly three values. For general sorting, use standard algorithms.

Common Mistakes to Avoid

Triplet Smaller: - Forgetting to add (`right - left`) when `sum < target` - Incrementing mid unnecessarily - Not sorting the array first

Dutch National Flag: - Incrementing mid after swapping with high - Not understanding why three regions work - Confusing the algorithm with quick-sort partitioning

Real Interview Scenarios

LeetCode References

- **LeetCode 259:** 3Sum Smaller (Medium)
- **LeetCode 75:** Sort Colors (Medium)
- **Related:** LeetCode 15 (3Sum), LeetCode 16 (3Sum Closest)

Companies Asking These Problems

- **Google** - Dutch National Flag (very common)
 - **Amazon** - Both problems
 - **Facebook/Meta** - 3Sum variants
 - **Microsoft** - Array manipulation problems
 - **Flipkart** - Sorting and arrangement problems
-

Recommended Practice Problems

1. **Two Pointer Foundation:**
 - Valid Palindrome
 - Reverse String
 - Two Sum II

2. **Build on Triplet Sum:**
 - 3Sum (LeetCode 15)
 - 3Sum Closest (LeetCode 16)
 - 4Sum (LeetCode 18)
 3. **Build on Dutch Flag:**
 - Sort Array by Parity
 - Partition Array by Pivot
 - Quickselect Algorithm
 4. **Mixed Pattern:**
 - Container with Most Water
 - Trapping Rain Water
 - Remove Duplicates from Sorted Array
-

Quick Reference

Triplet Sum Smaller Template

```
def countTriplets(arr, target):
    arr.sort()
    count = 0
    n = len(arr)

    for i in range(n-2):
        left, right = i+1, n-1
        while left < right:
            total = arr[i] + arr[left] + arr[right]
            if total < target:
                count += right - left
                left += 1
            else:
                right -= 1

    return count
```

Dutch National Flag Template

```
def sortDutchFlag(arr):
    low, mid, high = 0, 0, len(arr) - 1

    while mid <= high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 1:
            mid += 1
```

```

        mid += 1
    else:
        arr[mid], arr[high] = arr[high], arr[mid]
        high -= 1

    return arr

```

Key Takeaways

Triplet Sum Smaller

- **Sorted array + fixed element + two pointers = range counting**
- When $\text{sum} < \text{target}$: $\text{count} += (\text{right} - \text{left})$ captures all valid combinations
- **Time:** $O(n^2)$, **Space:** $O(1)$

Dutch National Flag

- **Three pointers** define **three regions** in array
- **Single pass** processing while maintaining invariants
- When $\text{mid} > \text{high}$: All elements are partitioned correctly
- **Time:** $O(n)$, **Space:** $O(1)$

General Pattern Learning

- **Sort** $\rightarrow$ **Two Pointer** is powerful for array problems
 - **Three Pointers** extends two-pointer to partition problems
 - **Interview success** = Pattern recognition + Implementation skill
-

References

- **LeetCode Problem Links:**
 - <https://leetcode.com/problems/3sum-smaller/> (259)
 - <https://leetcode.com/problems/sort-colors/> (75)
 - **Related Problems:**
 - LeetCode 15 (3Sum)
 - LeetCode 16 (3Sum Closest)
 - LeetCode 18 (4Sum)
-

Category: DSA Interview Preparation

Difficulty Level: Medium

Time to Understand: 2-3 hours of practice